ملزمة الويك 3

أسئلة محلولة — Software Design (الجابتر الخامس) من المصادر

حلّ الـ 12 MCQ + 24 سؤال مقالي، موثّقًا بنصوص Agarwal و Pressman و Mall وشرائح Aggarwal & Singh

شنو هذا الملف؟ حلّ كامل لأسئلة الجابتر الخامس: 12 سؤال اختيارات (MCQ 5.1–5.12) و24 سؤال مقالي (Exercises 5.1–5.24).

منين جت الحلول؟ مو تخمين ولا استنباط من شرائح فقط — كل جواب مسنود إلى **نص حقيقي** من الكتب الموجودة بالمجلد: - **Mall**, *Fundamentals of Software Engineering*, 4th ed. — الفصل 5 (التصميم) والفصل 7 (OOD). - **Pressman**, *Software Engineering: A Practitioner's Approach* — الفصل 13 (Design Concepts and Principles). - **Agarwal & Agarwal**, *Software Engineering and Testing*, 2010 — الفصل 5 (System Design). - **Singh**, *Software Engineering*, 3rd ed. — شرائح الجابتر الخامس (المصدر اللي جت منه الأسئلة).

كل جواب مصنّف: حفظ (نقلي — الجواب حرفيًا بالمصدر) · فهم (يحتاج شرح وربط) · تحليل (نقاش مبني على المصادر).

ملاحظة أمانة: إذا كانت إجابة الدكتور النموذجية (بالمحاضرة) تختلف عن الحل هنا، **إجابة الدكتور هي المرجع للامتحان** — لأن التصحيح إله.

القاعدة الذهبية للجابتر كله

فكرة واحدة يدور عليها الجابتر كامل: **التصميم الجيد = cohesion عالي + coupling واطي.**

- **Cohesion** (الترابط داخل الوحدة) ← كلما **زاد**، كلما أحسن.
- **Coupling** (الاعتماد بين الوحدات) ← كلما **قل**، كلما أحسن.
- النتيجة تُسمّى **Functional Independence** — وهي مفتاح أي تصميم جيد.

هذي الجملة مكتوبة صراحةً في Mall (ص 292): "*A design solution is said to be highly modular, if the different modules in the solution have high cohesion and their inter-module couplings are low.*"

خريطة الجابتر — منين تجي كل مجموعة أسئلة

القسم	الشرائح (Aggarwal)	المصدر النصي (الكتاب)	الأسئلة
What is Design	9–2	Mall ص 287–288	5.1–5.3
Modularity	14–10	Agarwal ص 125–127 · Pressman ص 343–345	5.4، 5.7

القسم	الشرائح (Aggarwal)	المصدر النصي (الكتاب)	الأسئلة
Module Coupling	24–15	Mall ص 298–299 · Pressman ص 354–355	5.5، 5.8، 5.9
Cohesion	31–25	Mall ص 295–298 · Pressman ص 353	5.6، 5.7، 5.8، 5.10
Strategy of Design	35–32	Pressman ص 348–349 · Mall ص 302	5.11، 5.12
Function-Oriented + Structure Chart	49–36	Agarwal ص 127–131	5.13–5.15
SDD (IEEE 1016)	57–50	Mall (تصميم المستند)	5.21، 5.23
Object-Oriented Design	80–58	Mall ص 388–405 · Pressman	5.16–5.20، 5.22
أسئلة شاملة	—	كل المصادر	5.24

الجزء الأول — Multiple Choice Questions (5.1 – 5.12)

الجزء الأسهل: الجواب موجود حرفيًا في الشرائح. الأرقام محفوظة كما في المصدر.

#	السؤال	الجواب	التصنيف	مصدر التحقق
5.1	The most desirable form of coupling is	**(b) Data Coupling**	حفظ	Aggarwal slide 19–20: محور Coupling يضع Data عند طرف *Best*؛ Pressman ص 354 يؤكد أن data coupling = "low coupling".
5.2	The worst type of coupling is	**(a) Content coupling**	حفظ	Aggarwal slide 19: Content عند طرف *Worst*؛ Pressman ص 355: "*The highest degree of coupling, content coupling ... can and should be avoided.*"
5.3	The most desirable form of cohesion is	**(c) Functional cohesion**	حفظ	Aggarwal slide 27: Functional عند طرف *(high) Best*؛ Mall ص 295: "*functional is the best cohesion possible.*"
5.4	The worst type of cohesion is	**(b) Coincidental cohesion**	حفظ	Aggarwal slide 27: Coincidental عند طرف *(low) Worst*؛ Mall ص 295: "*coincidental is the worst type of cohesion.*"
5.5	Which one is not a strategy for design?	**(c) Embedded design**	حفظ	Aggarwal slides 33–35: الاستراتيجيات الثلاث = Bottom-up, Top-down, Hybrid. "Embedded" ليست منها.
5.6	Temporal cohesion means	**(c) Cohesion with respect to time**	حفظ	Aggarwal slide 29: "*tasks that are related by the fact that all tasks must be executed in the same time-span.*"
5.7	Functional cohesion means	**(a) Operations are part of single functional task and are placed in same procedures**	حفظ	Aggarwal slide 28: "*A and B are part of a single functional task. This is very good reason for them to be contained in the same procedure.*"

#	السؤال	الجواب	التصنيف	مصدر التحقق
5.8	When two modules refer to the same global data area, they are related as	**(d) Common coupled**	حفظ	Mall + Aggarwal slide 21 ص 299: shared/global data = common coupling area.
5.9	The module in which instructions are related through flow of control is	**(c) Procedural cohesion**	فهم	Aggarwal slide 29: "*instructions … combined because there is a specific order in which the tasks are to be completed.*"
5.10	The relationship of data elements in a module is called	**(b) Cohesion**	فهم	Aggarwal slide 25: cohesion = degree to which elements of a module are functionally related. (Coupling يشير إلى العلاقة *بين* الوحدات، cohesion داخل الوحدة.)
5.11	A system that does not interact with external environment is called	**(a) Closed system**	حفظ	تعريف نظامي: النظام المغلق لا يتفاعل مع البيئة الخارجية؛ المفتوح يتفاعل.
5.12	The extent to which different modules are dependent upon each other is called	**(a) Coupling**	حفظ	Aggarwal slide 15: "*Coupling is the measure of the degree of interdependence between modules.*"

> **تنبيه على الترقيم:** في المصدر الأصلي ورد سؤال 5.11 قبل 5.10 (اختلاف ترتيب الشرائح). الجدول أعلاه مرتّب منطقيًا. عند الحفظ بالامتحان، ركّز على **المعنى** لا الرقم.

الجزء الثاني — Exercises (5.1 – 5.24) — حلول نموذجية

> كل سؤال: ① **السؤال كما في المصدر** ② **الشرح المفاهيمي (عربي)** ③ **الجواب النموذجي + الاستشهاد**. المصطلحات بالإنجليزية، الشرح بالعربي — حسب منهج المادة.

5.1 — What is design? Describe the difference between conceptual design and technical design.

التصنيف: فهم · **المصدر:** Aggarwal slides 2–7; Mall ص 287–288

الشرح المفاهيمي — تصميم شنو؟ التصميم هو مرحلة **"HOW"** بعد مرحلة **"WHAT"**. الـ Requirements تگول **شنو** يريد العميل؛ الـ Design يگول **شلون** نبنيه. التصميم *أكثر إبداعًا من التحليل* لأنه ما يتبع القواعد وحدها — يحتاج قرارات وحلول.

الجواب النموذجي: - **Design** = عملية تحويل المتطلبات إلى وصف للبنية والوحدات والواجهات والبيانات اللازمة للتنفيذ. مخرجُه مستند هو **SDD (Software Design Document)**. - **Conceptual design** (تصميم مفهومي) — يجيب أسئلة المستخدم: منين تجي البيانات؟ شنو يصير بالبيانات؟ شلون يبدو النظام للمستخدم؟ شنو الخيارات المطروحة؟ شنو توقيت الأحداث؟ - **Technical design** (تصميم تقني) — يوصف البنية التحتية: تهيئة العتاد، احتياجات البرمجيات، واجهات الاتصال، المدخلات/المخرجات، معمارية البرمجيات والشبكة، وأي شي يترجم المتطلبات إلى حل.

الاستشهاد: Aggarwal slide 5 يفصلها بمخاطَبين مختلفين: Conceptual design يخاطب **Customer** (What)، و Technical design يخاطب **System Builders** (How). و Mall ص 287–288 يذكر أن التصميم يمر بمرحلتين: **high-level design** (تحديد الوحدات وتفاعلاتها) ثم **detailed design** (تحديد الخوارزميات وبنى البيانات).

5.2 — Discuss the objectives of software design. How do we transform an informal design to a detailed design?

التصنيف: فهم · **المصدر:** Aggarwal slides 3، 8، 9

الشرح المفاهيمي: التصميم مو مجرد "شكل حلو" — له أهداف قابلة للفحص، وله مسار تحوّل من فكرة إلى تفصيل.

الجواب النموذجي — الأهداف (slide 8): 1. **complete & Correct** — يغطي كل المتطلبات بلا نقص ولا خطأ. 2. **Understandable** — مفهوم لمن سينفّذ ولمن سيصون. 3. **At the right level** — بمستوى التجريد المناسب للمرحلة. 4. **Maintainable** — يقبل التعديل لاحقًا بتكلفة معقولة.

التحوّل من informal إلى detailed (slide 3 + 9): المسار: informal design outline ← informal design ← more formal design ← finished design. الآلية: جمع بيانات متطلبات المستخدم ← تحليلها ← تصوّر تصميم عالي المستوى ← ثم **Refinement** تدريجيًا (كل خطوة تفصّل أكثر) ← ثم توثيق والتحقق ضد المتطلبات.

الاستشهاد: Pressman ص 342 يسمي هذه العملية **Stepwise Refinement** (Wirth): "*A program is developed by successively refining levels of procedural detail.*" ويؤكد أن التجريد (Abstraction) والتنقيح (Refinement) مفهومان **متكاملان**: الأول يخفي التفاصيل، والثاني يكشفها تدريجيًا.

5.3 — Do we design software when we "write" a program? What makes software design different from coding?

التصنيف: تحليل · **المصدر:** Aggarwal slide 2; Pressman ص 337، 351

الشرح المفاهيمي: فرّق بين **التصميم** و**الترميز (coding)**. ممكن ترمّز برنامج بلا تصميم — وهذا بالضبط مصدر الأزمات.

الجواب النموذجي: - التصميم يبدأ **قبل** الترميز، وهو قرارات عالية المستوى (البنية، الوحدات، الواجهات)، أما الترميز فينفّذ التفاصيل الدنيا فقط. - الفرق الأساسي = **مستوى التجريد (level of abstraction)**: التصميم أعلى تجريدًا من الكود المصدري.

الاستشهاد: Pressman ص 351: "*Design is not coding, coding is not design ... the only design decisions made at the coding level address the small implementation details.*" و slide 2 يسمّي التصميم "*More creative than analysis*" — أي كائن إبداعي، بينما الترميز تنفيذ.

5.4 — What is modularity? List the important properties of a modular system.

التصنيف: حفظ + فهم · **المصدر:** Aggarwal slides 10–13; Agarwal ص 125–126; Pressman ص 344

الشرح المفاهيمي: الوحدة (module) مو محددة بتعريف واحد — من روتين FORTRAN إلى صنف Java إلى مهمة مبرمج فردية. المعيار المشترك: **وحدات مُدارة جيدًا بواجهات مُعرّفة جيدًا**.

الجواب النموذجي — الخصائص (slide 12): 1. نظام فرعي (subsystem) مُعرّف جيدًا. 2. غرض (purpose) مُعرّف جيدًا. 3. قابل للترجمة والتخزين منفصلًا في مكتبة. 4. يمكن أن يستخدم وحدات أخرى. 5. أسهل استخدامًا مما هو بناءًا. 6. أبسط من الخارج منه من الداخل.

أهمية الـ modularity (slide 13): "*Modularity is the single attribute of software that allows a program to be intellectually manageable*" — يرفع وضوح التصميم، ويسهّل التنفيذ والتنقيح والاختبار والتوثيق والصيانة.

الاستشهاد: Agarwal ص 125 يضيف خصائص مستقرة: (1) كل دالة لها غرض واحد مُعرّف، (2) كل دالة تتعامل مع بنية بيانات رئيسية واحدة على الأكثر، (3) الوحدات تتشارك البيانات الشاملة بشكل انتقائي، (4) الدوال التي تتعامل مع ADT تكون مغلّفة مع بنية البيانات (encapsulated).

5.5 — Define module coupling and explain different types of coupling.

التصنيف: حفظ + فهم · **المصدر:** Aggarwal slides 15–20; Mall ص 298–299; Pressman ص 354–355

الشرح المفاهيمي: Coupling = مقياس الاعتماد بين وحدتين. القاعدة: **كل ما قلّ، أحسن**. Ye، وهي مبنيّة على **تعقيد الواجهة**: عدد المعاملات وتعقيدها.

الأنواع من الأفضل إلى الأسوأ (slide 19 + Mall ص 298–299 + Pressman ص 354–355):

#	النوع	التعريف	التقييم
1	**Data coupling**	التواصل بتمرير عنصر بيانات أولي (int, float, char).	الأفضل
2	**Stamp coupling**	تمرير بنية بيانات كاملة (record/structure) بدل عنصر واحد.	جيد
3	**Control coupling**	تمرير معلومة تحكّم (flag) توجّه تسلسل التنفيذ في الوحدة الأخرى.	متوسط
4	**External coupling**	ارتباط الوحدة ببيئة خارجية (I/O، أجهزة، بروتوكولات).	عالٍ
5	**Common coupling**	تشارك بيانات شاملة (global data area) بين وحدات عدة.	عالٍ
6	**Content coupling**	وحدة تغيّر بيانات وحدة أخرى، أو ينتقل التحكم إلى منتصف وحدة.	الأسوأ

الاستشهاد: Mall ص 298: "*The degree of coupling between two modules depends on their interface complexity.*" و Pressman ص 355 عن محتوى-التكرار: "*The highest degree of coupling, content coupling, ... can and should be avoided.*"

مثال تطبيقي (slide 18): نظام سجلات طلاب — تمرير "اسم الطالب + رقمه + العنوان + المقرر" (tight coupling) **سيئ**؛ تمرير "رقم الطالب" فقط (loose coupling) **جيد**.

5.6 — Define module cohesion and explain different types of cohesion.

التصنيف: حفظ + فهم · **المصدر:** Aggarwal slides 25–30; Mall ص 295–298; Pressman ص 353

الشرح المفاهيمي: Cohesion = مقياس القوة الوظيفية داخل الوحدة (مدى ارتباط عناصرها لهدف واحد). القاعدة: **كل ما زاد، أحسن**.

الأنواع من الأفضل إلى الأسوأ (slide 27 + Mall ص 295–298):

#	النوع	التعريف
1	**Functional**	كل دوال الوحدة تتعاون لإنجاز **مهمة واحدة** (الأفضل).
2	**Sequential**	مخرج دالة هو مدخل الدالة التالية في التسلسل.

#	النوع	التعريف
3	**Communicational**	كل الدوال تعمل على **نفس بنية البيانات**.
4	**Procedural**	الدوال تُنفَّذ واحدًا بعد الأخرى بترتيب محدد (بأغراض مختلفة).
5	**Temporal**	الدوال مرتبطة بأنها تُنفَّذ **في نفس الفترة الزمنية** (initialise/setup/shutdown).
6	**Logical**	الدوال تنتمي إلى **نفس الصنف المنطقي** (كل الإخراج، كل معالجة الأخطاء).
7	**Coincidental**	عناصر لا علاقة بينها — تجميع عشوائي (الأسوأ).

اختبار سريع للتعيين (Mall ص 297): جرّب تصف الوحدة بجملة واحدة — إذا احتجت "first/next/then" ⇒ procedural/sequential؛ إذا احتجت "initialise/setup/shutdown" ⇒ temporal؛ إذا نجحت بجملة واحدة ⇒ functional.

الاستشهاد: Mall ص 297 عن functional cohesion: "*When a module possesses functional cohesion, then we should be able to describe what the module does using only one simple sentence.*"

5.7 — Discuss the objectives of modular software design. What are the effects of module coupling and cohesion?

التصنيف: فهم · **المصدر:** Aggarwal slide 31; Mall ص 294–295 (Functional Independence)

الشرح المفاهيمي: الهدف النهائي من التصميم الموديولي هو **الاستقلال الوظيفي (Functional Independence)** — وهو نتيجة مباشرة لـ cohesion عالي + coupling واطي.

الجواب النموذجي — الفوائد الثلاث (Mall ص 294–295): 1. **Error isolation** — الوحدة المستقلة فشلها لا ينتشر؛ وتحديد الخطأ أسهل. 2. **Scope of reuse** — الوحدة المستقلة قابلة للسحب وإعادة الاستخدام في برامج أخرى. 3. **Understandability** — الوحدات تُفهم منعزلًا، فتعقيد التصميم يقل.

تأثير coupling/cohesion: تجعل النظام **قابلًا للفهم والصيانة والتوسعة**؛ على العكس، الوحدات المتشابكة (high coupling) تؤدي إلى **ripple effect** — خطأ واحد ينتشر بالنظام.

الاستشهاد: Pressman ص 352: "*Functional independence is a key to good design, and design is the key to software quality.*" والمصدر يعرّف الوحدة المستقلة بأنها "*single-minded*" قابلة للوصف بجملة بسيطة (subject-predicate-object).

5.8 — If a module has logical cohesion, what kind of coupling is this module likely to have with others?

التصنيف: تحليل · **المصدر:** Mall ص 296، 298؛ Aggarwal slides 21، 30

الشرح المفاهيمي: سؤال يربط بين الجانبين. الوحدة ذات **logical cohesion** تجمع دوالًا مختلفة (كلها إدخال، كلها إخراج، كلها معالجة أخطاء). هذا التجميع المفتعل يفرض عليها **معاملات تحكّم** لتمييز أي دالة تُنفَّذ — وهذا يخلق **control coupling**.

الجواب النموذجي: الوحدة ذات logical cohesion غالبًا تُظهر **Control coupling** (بل وقد ترتقي إلى **Common coupling** إذا اعتمدت على بيانات شاملة تُحدّد الوضع)، لأن تمييز الوظيفة المطلوبة يحتاج **flag** يُمرَّر ويُفحَص — وهذا تعريف control coupling بالضبط.

الاستشهاد: Aggarwal slide 21: "control coupling = *accomplished by means of flags … passing of control information*". والقاعدة العامة (slide 31): كلما انخفض cohesion، ارتفع coupling المتوقع.

5.9 — What problems are likely to arise if two modules have high coupling?

التصنيف: فهم · **المصدر:** Aggarwal slide 31; Mall ص 294; Pressman ص 354، 355

الجواب النموذجي — المشاكل: 1. **Ripple effect** — خطأ في وحدة ينتشر لأخرى (ذكرها Pressman ص 354). 2. **صعوبة التنقيح (debugging)** — تحديد مصدر الخطأ يصير مكلفًا؛ Pressman يعطي مثالًا (ص 355): كتلة k تنهار، والسبب الحقيقي كتلة g عدّلت بيانات شاملة. 3. **صعوبة إعادة الاستخدام** — الوحدة المترابطة يصعب سحبها لبرنامج آخر (Mall ص 294). 4. **هشاشة وحدة القياس** — أي تعديل بسيط قد يسبب "death of the project" (Aggarwal slide 31): تعديلات تافهة ظاهريًا قد تقتل المشروع). 5. **صعوبة الفهم** — تعقيد الواجهات يرفع الحمل الذهني.

الاستشهاد: Pressman ص 354: "*Highly coupled systems lead to debugging nightmares. Avoid them.*"

5.10 — What problems are likely to arise if a module has low cohesion?

التصنيف: فهم · **المصدر:** Mall ص 295–296; Pressman ص 353

الجواب النموذجي — المشاكل: 1. **ازدياد الاقتران** — خفض cohesion يرفع coupling: نقل أي دالة من الوحدة يجعل الدوال تتفاعل عبر وحدتين (Mall ص 298 آخر الفقرة). 2. **error propagation** — الوحدة غير المتجانسة تجمع وظائف مستقلة، فأي تعديل على إحداها قد يضرّ الباقي (Pressman ص 353 يعطي مثال وحدة معالجة الأخطاء التي تؤدي 5 مهام مستقلة). 3. **فقدان قابلية الفهم وإعادة الاستخدام** — لا يمكن وصف الوحدة بجملة واحدة، فتصعب صيانتها ونقلها. 4. **صعوبة الاختبار** — الوحدة تؤدي وظائف متباعدة، فتصميم حالات الاختبار يتشعّب.

الاستشهاد: Pressman ص 353: "*Combining the functions into a single module can serve only to increase the propagation likelihood of error.*"

5.11 — Describe the various strategies of design. Which design strategy is most popular and practical?

التصنيف: فهم + تحليل · **المصدر:** Aggarwal slides 33–35; Pressman ص 348–349

الجواب النموذجي — الاستراتيجيات الثلاث: 1. **Top-down design** — تبدأ بتحديد الوحدات الرئيسية ثم تفكيكها تدريجيًا (stepwise refinement) حتى الوصول لمستوى الجملة. الوحدات تصير متخصصة جدًا، كل وحدة يستخدمها أب واحد. 2. **Bottom-up design** — تبدأ بوحدات صغيرة (workers) تُجمع في **مكتبة** ثم تُبنى فوقها الوحدات الأعلى. 3. **Hybrid design** — الجمع بين الاثنين.

الأكثر شيوعًا وعملية = Hybrid. السبب (slide 35): النهج الـ top-down وحده لا يكفي — تحتاج bottom-up لـ: (1) السماح بوحدات فرعية مشتركة، (2) الاختبار من الأسفل حيث الحدس أبسط وعدد الوحدات أكبر، (3) استخدام وحدات مكتبة جاهزة (reuse).

الاستشهاد: Pressman ص 348–349 (Structural Partitioning) يؤكد المنطق نفسه: التقسيم الأفقي يوزّع الفروع، والتقسيم الرأسي يوزّع **التحكم في الأعلى والعمل في الأسفل** — "*Top-level modules should perform control functions and do little actual processing work.*"

5.12 — If some existing modules are to be re-used in building a new system, which design strategy is used and why?

التصنيف: تحليل · **المصدر:** Aggarwal slide 35; Pressman ص 348

الجواب النموذجي: تُستخدم **Bottom-up (أو Hybrid)**. السبب: الوحدات المكتبية (library modules) الجاهزة تُبنى **من الأسفل للأعلى** — تُجمّع الوحدات الصغيرة لتشكيل بنية أعلى. slide 35 يذكر صراحةً أن "استخدام وحدات مكتبة مكتوبة مسبقًا، وبالأخص إعادة استخدام الوحدات" من أسباب الحاجة للـ bottom-up.

الاستشهاد: Pressman ص 348: التقسيم الرأسي يسمح بإعادة استخدام وحدات workers منخفضة المستوى لأن تغييرها أقل propagation. وMall ص 295 عن **Scope of reuse**: الوحدة المستقلة "*can therefore be easily taken out and reused in a different program*".

5.13 — What is the difference between a flow chart and a structure chart?

التصنيف: فهم · **المصدر:** Aggarwal slides 40–43; Agarwal ص 127–129

الشرح المفاهيمي: كلاهما رسم، لكن يخدمان غرضين مختلفين تمامًا.

الجواب النموذجي:

وجه المقارنة	Flow chart	Structure chart
يمثّل	**تسلسل الخطوات والتحكم** (منطق الخوارزمية)	**التنظيم الهرمي للوحدات** واستدعاءاتها
المستوى	تفصيلي (خطوة بخطوة)	معماري (بنية النظام)
الأبعاد	تسلسل زمني / منطقي	شجرة تدرّج (root واحد)
الغرض	تصميم المنطق	توثيق التقسيم (decomposition) والواجهات

الاستشهاد: Agarwal ص 127: "*design to document hierarchical Structure charts are used during architectural parameters, structures, and interconnections in a system.*" وله **root واحد فقط** في أي structure chart. بينما flow chart يمثّل منطق التنفيذ. (slide 42 يعرض رموز structure chart: مربعات، أسهم تحكم، أسهم تدفق بيانات، وحدات مكتبية، اختيار مثلثي، تكرار.)

5.14 — Explain why it is important to use different notations to describe software designs.

التصنيف: تحليل · **المصدر:** Aggarwal slides 40، 46؛ Agarwal ص 127–131

الجواب النموذجي: لكل مُشاهد (stakeholder) احتياج مختلف، ولكل جانب من التصميم تمثيل أنسب: - **Structure chart** — للبنية الهرمية والواجهات. - **Pseudocode (PDL)** — لتوصيف المنطق التفصيلي؛ سهل التحويل لكود وسهل التعديل (Agarwal ص 130). - **Data flow diagrams / Data dictionaries** — للبيانات وتدفقها.

استخدام عدة صيغ يخفض **الغموض**، ويخدم جماهير مختلفة (مبرمج، مختبِر، مصمم)، ويجعل **المراجعة** أدق لأن كل تمثيل يكشف نوعًا مختلفًا من الأخطاء.

الاستشهاد: slide 40: التمثيل يكون "*graphically or mathematically*" عبر DFD, Data Dictionary, Structure Charts. Pseudocode وAgarwal ص 130 يبيّن أن pseudocode له ميزة "*Converting a pseudo-code to a programming language is much easier*".

5.15 — List a few well-established function oriented software design techniques.

التصنيف: حفظ · **المصدر:** Mall ص 302

الجواب النموذجي — خمس تقنيات معتمدة: 1. **Structured design** — Yourdon & Constantine (1979). 2. **Jackson's structured design** — Jackson (1975). 3. **Warnier-Orr methodology** — (1977, 1981). 4. **Step-wise refinement** — Wirth (1971). 5. **Hatley and Pirbhai's Methodology** — (1987).

الاستشهاد: Mall ص 302 يسرد هذه الخمس حرفيًا كـ *well-established function-oriented design approaches*. (ملاحظة: slide 36 يعرّف Function-Oriented design بأنه تفكيك النظام إلى وحدات متحاورة، كل وحدة بدالة مُعرّفة — منظور وظيفي لا كائني.)

5.16 — Define the following terms: Objects, Message, Abstraction, Class, Inheritance and Polymorphism.

التصنيف: حفظ + فهم · **المصدر:** Mall ص 388–405; Aggarwal slides 60–69

الجواب النموذجي — التعريفات:

المصطلح	التعريف	الاستشهاد
Object	كيان يحفظ حالة (state) ويعرض عمليات (behavior) لفحص/تعديل الحالة.	slide 60
Message	وسيلة تواصل بين الكائنات: هوية الهدف + اسم العملية المطلوبة + معاملاتها.	slide 61
Abstraction	إزالة غير المهم وتضخيم الجوهري (amplification of + elimination of irrelevant) essentials).	slide 61
Class	مجموعة كائنات تشارك بنية وسلوكًا مشتركين — تعمل كـ blueprint للكائنات.	slide 62; Mall ص 389
Inheritance	تعريف صنف جديد بتوسعة صنف قائم؛ الأصل = base/superclass، الجديد = derived/subclass. علاقة "is-a".	slide 66–68; Mall ص 390
Polymorphism	الكائن يتصرف بشكل مختلف عند استدعاء نفس العملية، حسب الكائن الفعلي المرتبط.	slide 69; Mall ص 402

إضافة (Mall ص 401–402): Encapsulation = إخفاء البيانات داخليًا وعدم الوصول إليها إلا بالدوال (تُسمى Information Hiding)، وفائدتها: الحماية من الوصول غير المصرّح، إخفاء البيانات، والاقتران الضعيف.

تقسيم Polymorphism (Mall ص 402–403): Static (overloading — يُحدّد وقت الترجمة) و**Dynamic** (overriding — يُحدّد وقت التشغيل).

5.17 — What is the relationship between abstract data types and classes?

التصنيف: فهم · **المصدر:** Mall ص 389، 403

الجواب النموذجي: كل class هو ADT، لكن ليس كل ADT هو class. - الـ ADT = نوع بيانات معرّف من المستخدم يخفي التمثيل الداخلي ويسرد العمليات المسموحة (مثل stack بطريقة تنفيذها المخفية). - الصنف (class) يحقق ذلك، **وإضافةً** يدعم **الوراثة** وبقية خصائص الكينونة (مثل polymorphism). - لذلك اللغة التي تدعم ADT لكن لا تدعم الوراثة تُسمّى **object-based** وليست **object-oriented** (مثل Ada).

الاستشهاد: Mall ص 389: "*since to be a ,an ADT need not be a class ,that a class is an ADT. But …It can be inferred*". وMall ص 392: "*If a language supports ADTs ,but does not support inheritance ,then it is called an object-based language*".

5.18 — Can we have inheritance without polymorphism? Explain.

التصنيف: تحليل (فخ) · **المصدر:** Mall ص 389، 392، 402

الشرح المفاهيمي: كثير يخلطون بينهما لأنهما دائمًا معًا. لكن الفرق مهم: **الوراثة إعادة استخدام (reuse)**، و**تعددية الشكل اختبار ديناميكي (dynamic dispatch)**. واحد ما يلزم الثاني.

الجواب النموذجي: نعم، ممكن. - الوراثة = آلية إعادة استخدام: صنف مشتق يورث بيانات ودوال الأصل ويضيف/يعيد تعريف. - Polymorphism = قدرة استدعاء واحد أن يُحلّ (bind) لتنفيذات مختلفة. - ممكن يكون عندك وراثة (استخدمت إعادة الاستخدام) بلا overriding، فلا يوجد dynamic binding. - العكس هو الممنوع: من تعريف slide 69، الـ polymorphic operation يترك التنفيذ للأصناف المشتقة — فهو **يفترض وجود وراثة**. لكن الوراثة وحدها لا تستلزم polymorphism.

الاستشهاد: Mall ص 392 عن الوراثة: الآلية تُوفّر "*code reuse and simplicity of program design*". وMall ص 403 يربط dynamic binding بالوراثة + Assignment) **Method overridden in a class hierarchy**) — أي التعددية تحتاج وراثة، لا العكس.

5.19 — Discuss the reasons for improvement using object-oriented design.

التصنيف: تحليل · **المصدر:** Mall ص 304، 307، 402؛ Aggarwal slides 58، 69

الجواب النموذجي — أسباب التحسّن في OOD: 1. **Encapsulation (data hiding)** — البيانات تُخبَّأ داخل الكائن ⇒ أخطاء متموضعة يسهل تتبعها (Mall ص 304). 2. **low coupling + High cohesion** — "*An ADT-based design displays high object-oriented designs are highly modular ,coupling. Therefore cohesion and low*" (Mall ص 304). 3. **Weak coupling between objects** — لأن الكائنات لا تغيّر بيانات بعضها مباشرة ⇒ فهم أسهل (Mall ص 402). 4. **Reuse & maintenance** — dynamic binding يسمح بإضافة أصناف مشتقة جديدة بأقل تغيير (Mall ص 405)، وجعل الكود أوجز وأقرب للفهم. 5. **Abstraction** — إدارة التعقيد بإخفاء غير المهم (slide 61، Mall ص 304).

الاستشهاد: Mall ص 304 يسرد الفوائد الثلاث لـ ADT: encapsulation يُموضع الأخطاء، تصميم ADT عالي cohesion واطي coupling، والتجريد يسهّل الفهم. وslide 58 يعرّف OOD بأنه "*orthogonal to function oriented design*".

5.20 — Explain the design guidelines that can be used to produce "good quality" classes or reusable classes.

التصنيف: تحليل · **المصدر:** Mall ص 304، 401، 405؛ Pressman ص 374

الجواب النموذجي — إرشادات الأصناف الجيدة/القابلة لإعادة الاستخدام: 1. **تغليف البيانات (Encapsulation)** — اجعل بيانات الكائن خاصة، والوصول فقط عبر الدوال ⇒ يحمي من التعدي ويسمح بالصيانة (Mall ص 401). 2. **cohesion عالي و coupling واطي** — الصنف يؤدي مسؤولية واحدة واضحة (Mall ص 304). 3. **استخدم الوراثة لإعادة الاستخدام** — ضع المشترك في base class (Mall ص 392). 4. **فضّل الربط الديناميكي (dynamic binding)** — يسمح بتوسيع النظام بإضافة أصناف دون تعديل الكود القائم (Mall ص 405). 5. **حافظ على واجهة بسيطة وثابتة** — الواجهات القليلة والبسيطة تسهّل إعادة الاستخدام (Pressman ص 374: continuity + modular understandability).

الاستشهاد: Mall ص 401 عن Data Hiding: "*other ,are modified if the internal data or the method body of a class maintenance and bug correction classes are not affected. This leads to easier.*"

5.21 — List the points of a simplified design process.

التصنيف: حفظ · **المصدر:** Aggarwal slide 3 (Design framework)

الجواب النموذجي — خطوات عملية التصميم المبسطة (slide 3): 1. جمع بيانات متطلبات المستخدم (Gather data on user requirements). 2. تحليل بيانات المتطلبات (Analyze requirements data). 3. تصوّر تصميم عالي المستوى (Conceive of a high level design). 4. تنقيح وتوثيق التصميم (document the design & Refine). 5. الحصول على أجوبة أسئلة المتطلبات (Obtain answers to requirement questions). 6. التحقق من التصميم مقابل المتطلبات (Validate the design against the requirements) ⇒ مخرج: **Completed design**.

الاستشهاد: Aggarwal Figure 1 (slide 3) يعرض المخطط كاملًا. ويقابله Mall ص 287 (high-level ثم detailed) وPressman ص 338 (التصميم عملية تكرارية تُترجم المتطلبات إلى **blueprint**).

5.22 — Discuss the differences between object oriented and function oriented design.

التصنيف: تحليل (مقارنة امتحانية كلاسيكية) · **المصدر:** Mall ص 303–307; Aggarwal slide 58

الجواب النموذجي — جدول المقارنة:

وجه المقارنة	Function-oriented	Object-oriented
الوحدة الأساسية	الدوال والوحدات (functions/modules)	الكائنات (objects)
محور التفكيك	ممثّل بالمصطلحات: **verbs** (أفعال)	(أسماء) **nouns**
حالة النظام (state)	**مركزية** (global shared data)	**موزّعة** بين الكائنات (decentralised)
الوصول للبيانات	عبر بيانات شاملة مشتركة	عبر دوال الكائن فقط (encapsulation)
الأساس التجريدي	الخدمات المتاحة للمستخدم (issue-book...)	كيانات العالم الواقعي (member, book...)
طريقة التجميع	تجميع الدوال التي تشكّل دالة أعلى	تجميع الدوال حسب البيانات التي تعمل عليها

الاستشهاد: Mall ص 305، 307 يعرض الفروق بأمثلة (نظام إنذار الحريق، ونظام الرواتب). وslide 58: OOD يركّز على البيانات لا الدوال، لذا هو "*orthogonal*" (متعامد) معه.

5.23 — What documents should be produced on completion of the design phase?

التصنيف: حفظ + فهم · **المصدر:** Aggarwal slides 50–57 (IEEE 1016-1998)؛ Mall ص 287

الجواب النموذجي: المخرج الرسمي هو **Software Design Description — SDD** (الموصى به في IEEE STD 1016-1998). محتواه: - **Scope** + **References** (يستشهد بـ IEEE 830-1998 للـ SRS و IEEE 610.12-1990 للتفاعلات). - **Design entities** و **design entity attributes** — (slide 53): Identification, Type, Purpose, Function, Subordinates, Dependencies, Resources, Interface, Processing, Data. - **Design views** (slide 57): Decomposition description، Dependency description، Interface description، Detail description.

أهمية الـ SDD: هو **blueprint** للنظام — يمثّل تحويل الـ SRS إلى وصف بنية البرمجيات والواجهات والبيانات اللازمة للتنفيذ.

الاستشهاد: slide 52: "*The SDD shows how the software system will be structured to satisfy the requirements identified in the SRS ... Hence, SDD becomes the blueprint for the implementation activity.*" وPressman ص 366 يسمّي مخرج التصميم "*a blueprint for constructing the software*".

5.24 — Can a system ever be completely "decoupled"? That is, can the degree of coupling be reduced so much that there is no coupling between modules?

التصنيف: تحليل (فلسفي) · **المصدر:** Aggarwal slide 15 (Uncoupled)؛ Mall ص 292–294; Pressman ص 354

الشرح المفاهيمي: slide 15 يعرض ثلاث حالات: **Uncoupled** (بلا اعتماد)، **Loosely coupled**، **Highly coupled**. فمن الناحية المثالية، الـ "uncoupled" موجودة كحالة نظرية.

الجواب النموذجي: نظريًا **نعم**، الحالة "uncoupled" يُشار إليها في المصدر (slide 15a)، لكن **عمليًا لا** يمكن لنظام نافع أن يكون بلا اقتران إطلاقًا — لأن وظيفة النظام الواحدة تتطلب أن تتعاون الوحدات. - وحدة بلا أي اقتران = وحدة معزولة لا تساهم في سلوك النظام ⇒ غير ذات جدوى. - الحد العملي هو **data coupling** (أقل اقتران مفيد): الوحدات تتواصل بالحد الأدنى من البيانات الأولية اللازمة. - الحل: تقليل الاقتران إلى **أدنى حد مفيد**، لا إلى الصفر.

الاستشهاد: slide 15 + Mall ص 292: "*Two modules either do not interact with each other at all or at best they are said to have low coupling, or only a few primitive data items interact by passing no data.*" — أي أن أفضل حالة **عملية** هي "low coupling" وليس "zero coupling". وPressman ص 354 يشدّد: "*In software design, we strive for the lowest possible coupling*" — lowest، لا none.

ملحق — جدول MCQs المرجعي السريع (للحفظ)

الرقم	الجواب	المفهوم المختبر
5.1	(b) Data Coupling	أفضل coupling
5.2	(a) Content coupling	أسوأ coupling
5.3	(c) Functional cohesion	أفضل cohesion

الرقم	الجواب	المفهوم المختبر
5.4	(b) Coincidental cohesion	cohesion أسوأ
5.5	(c) Embedded design	ليست استراتيجية
5.6	(c) Cohesion with respect to time	temporal
5.7	(a) single functional task / same procedures	functional
5.8	(d) Common coupled	global data
5.9	(c) Procedural cohesion	flow of control
5.10	(b) Cohesion	relationship of data elements within a module
5.11	(a) Closed system	لا يتفاعل مع البيئة
5.12	(a) Coupling	degree of interdependence

Retrieval set — أسئلة استرجاع فعّال (الجواب محجوب)

غطِّ العمود الأيمن وحاول الجواب من ذاكرتك، ثم اكشف. هذي أقوى طريقة تثبيت قبل الامتحان.

السؤال	الجواب
شنو أفضل نوع coupling؟	Data coupling
شنو أسوأ نوع coupling؟	Content coupling
شنو أفضل نوع cohesion؟	Functional cohesion
شنو أسوأ نوع cohesion؟	Coincidental cohesion
شنو الاستراتيجية الأكثر عملية؟	Hybrid design
شنو الفرق بين flow chart و structure chart؟	flow = منطق/تسلسل ؛ structure = تنظيم هرمي للوحدات
كم تقنية function-oriented معتمدة ذكرها Mall؟	5 (Structured, Jackson, Warnier-Orr, Step-wise refinement, Hatley-Pirbhai)
هل ممكن وراثة بلا polymorphism؟	نعم — والسبب: التعددية تحتاج وراثة، لا العكس
شنو مخرج مرحلة التصميم الرسمي؟	SDD (IEEE 1016)
العلاقة بين class و ADT؟	كل class هو ADT، وليس كل ADT class

نهاية الملّزمة. المصادر النصية: Mall 4th (Ch.5, Ch.7) · Pressman (Ch.13) · CS Agarwal 2010 (Ch.5) · Singh & Aggarwal 3rd (slides Ch.5).